Day 96 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 14, Day 6 reference

Watch: Fine-tuning Large Language Models Explained (AIHub101) | GPT Models Explained: How Fine-Tuning Changes Everything! (Super Data Science)

`day96_fine_tuning.py`, segment by segment, in the order `main()` uses things. Day 90's model, Day 92's schedule and measurement, and Day 93's loader and generator are imported and were explained on their own days. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: θ = a weight · K = the trainable weights · η_t = learning rate at step t · m, v = Adam's moving averages · bits = held-out bits per character

Segment 0 — Setup

Imports and settings

CODE

```
with contextlib.redirect_stdout(io.StringIO()):
    import day92_training_mini_gpt as d92
    import day93_sampling_strategies as d93
d90 = d92.d90
START_MARK = "Master:\nBoatswain!"
SPLITS = (0.70, 0.15, 0.15)
STEPS, CHECK_EVERY, WARMUP = 400, 25, 20
FT_LRS = (1e-4, 3e-4, 1e-3)
SCRATCH_LR = 3e-3
TOP = ("blocks.3.", "Wout", "bout")
SMALL_FRAC = 0.25
N_PROMPTS, GEN_LEN = 20, 300
```

- `import day92_training_mini_gpt as d92` — Day 92's script as a module: its functions become d92.lr_schedule, d92.eval_bits.
- `TOP = ("blocks.3.", "Wout", "bout")` — name prefixes of the 'top only' weights: the last block and the output layer.
- `FT_LRS = (1e-4, 3e-4, 1e-3)` — 1e-4 is scientific notation for 0.0001.

Segment 1 — The task

task_data and ids: The Tempest, split in order

CODE

```
def task_data():
    tr_text, va_text = d90.load_text()
    s = va_text.index(START_MARK)
    play, rest = va_text[s:], va_text[:s]
    a, b = int(SPLITS[0] * len(play)), int((SPLITS[0] + SPLITS[1]) * len(play))
    return tr_text, play[:a], play[a:b], play[b:], rest
def ids(text, vocab):
    ix = {c: i for i, c in enumerate(vocab)}
    return np.array([ix[c] for c in text])
```

- `va_text.index(START_MARK)` — the position where the play's first lines begin in the held-out text.
- `play[:a], play[a:b], play[b:]` — train, dev, test in the play's own order: no shuffling, so the test part is truly later text.

- `{c: i for i, c in enumerate(vocab)}` — a dictionary comprehension: character to id.

Segment 2 — Fine-tuning

adam_subset: Adam on the trainable weights only

MATH

$$m \leftarrow 0.9\,m + 0.1\,g,\quad v \leftarrow 0.999\,v + 0.001\,g^2,\quad \theta \leftarrow \theta - \eta\,(m / (1 - 0.9^t)) / (\sqrt{(v / (1 - 0.999^t))} + \varepsilon), \text{ for } \theta \text{ in } K$$

CODE

```
def adam_subset(P, G, state, lr, keys, clip=1.0):
    g = {k: v for k, v in d90.w12.leaves(G) if k in keys}
    scale = min(1.0, clip / np.sqrt(sum(np.sum(a ** 2) for a in g.values())))
    state["t"] += 1; t = state["t"]
    for k, A in d90.w12.leaves(P):
        if k not in keys:
            continue
        m = state["m"].setdefault(k, np.zeros_like(A)); v = state["v"].setdefault(k,
            np.zeros_like(A))
        gk = g[k] * scale
        m[:] = 0.9 * m + 0.1 * gk; v[:] = 0.999 * v + 0.001 * gk ** 2
        A -= lr * (m / (1 - 0.9 ** t)) / (np.sqrt(v / (1 - 0.999 ** t)) + 1e-8)
```

- `if k not in keys: continue` — frozen weights are skipped: no update and no Adam state.
- `state["m"].setdefault(k, np.zeros_like(A))` — creates the moving average the first time a key is seen, then reuses it.
- `m[:] = ...` — writes into the existing array in place, so the stored state changes too.
- `A -= ...` — an in-place update: the model's own array changes, not a copy.

snapshot: a deep copy for the best checkpoint

CODE

```
def snapshot(P):
    return {k: (snapshot(v) if isinstance(v, dict) else v.copy()) for k, v in P.items()}
```

- `snapshot(v) if isinstance(v, dict)` — recursion: a nested dictionary is copied by calling the same function on it.
- `v.copy()` — a new array with the same numbers, so later in-place updates do not reach it.

fine_tune: one run, start to best checkpoint

CODE

```
def fine_tune(job):
    name, init, which, lr, frac, replay, seed = job
    tr_text, ft_tr, ft_dev, ft_te, rest = task_data()
    P0, vocab, cfg = d93.load_model()
    T, B, h = cfg["T"], cfg["B"], cfg["h"]
    tr, dev, te, gen = (ids(x, vocab) for x in (ft_tr, ft_dev, ft_te, rest))
    tr = tr[:int(len(tr) * frac)]
    old = ids(tr_text, vocab)
    with threadpool_limits(1, "blas"):
        rng = np.random.RandomState(seed)
        P = P0 if init == "pretrained" else d90.init_gpt(rng, len(vocab), d=cfg["d"],
            ff=cfg["ff"], layers=cfg["layers"], t=T)
        keys = {k for k, _ in d90.w12.leaves(P) if which == "all" or k.startswith(TOP)}
        state = {"t": 0, "m": {}, "v": {}}
        best = (d92.eval_bits(P, dev, T, h), 0, snapshot(P))
        curve = [(0, best[0])]
        t0 = time.perf_counter()
        for step in range(1, STEPS + 1):
            o = rng.randint(0, len(tr) - T - 1, B)
            x = np.stack([tr[i:i + T] for i in o]); y = np.stack([tr[i + 1:i + T + 1] for i in o])
            if replay:
                o2 = rng.randint(0, len(old) - T - 1, B // 2)
                x[B // 2:] = np.stack([old[i:i + T] for i in o2]); y[B // 2:] = np.stack([old[i +
                    1:i + T + 1] for i in o2])
            _, G = d90.gpt_loss_grads(x, y, P, h=h)
            adam_subset(P, G, state, d92.lr_schedule(step - 1, lr, warmup=WARMUP, total=STEPS),
                keys)
            if step % CHECK_EVERY == 0:
                d = d92.eval_bits(P, dev, T, h); curve.append((step, d))
                if d < best[0]:
                    best = (d, step, snapshot(P))
        Pb = best[2]
        return dict(name=name, curve=np.array(curve), best_step=best[1], dev=best[0],
            test=d92.eval_bits(Pb, te, T, h),
                    general=d92.eval_bits(Pb, gen, T, h), trainable=sum(A.size for k, A in
                        d90.w12.leaves(P) if k in keys),
                    minutes=(time.perf_counter() - t0) / 60, P=Pb if name == "full fine-tuning"
                        else None)
```

- `name, init, which, lr, frac, replay, seed = job` — tuple unpacking: one job is a 7-tuple, so it can be sent to a worker process.
- `x[B // 2:] = ...` — replay overwrites the second half of the batch with windows from the original training text.
- `best = (d, step, snapshot(P))` — the best dev loss so far, its step and a deep copy of the weights.
- `P=Pb if name == "full fine-tuning" else None` — only one run sends its weights back: they are pickled between processes.

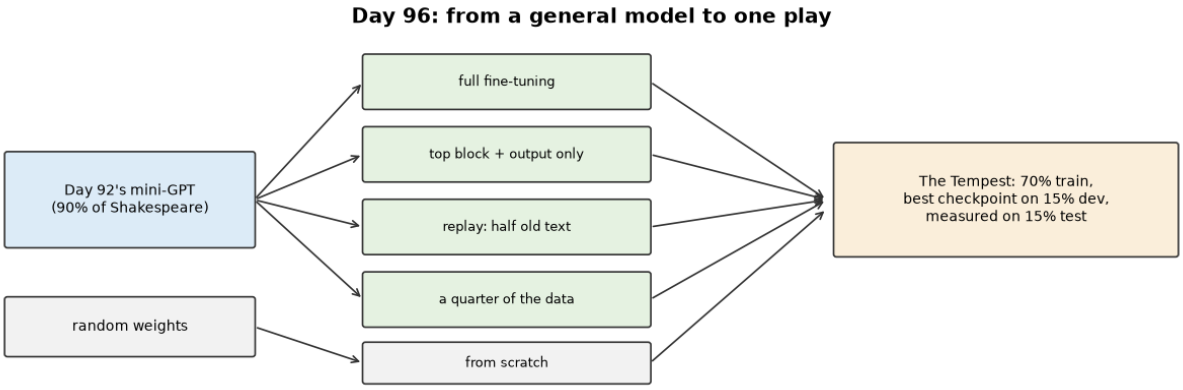
run_parallel: one process per run

CODE

```
def run_parallel(jobs):
    with mp.get_context("spawn").Pool(len(jobs)) as pool:
        return pool.map(fine_tune, jobs)
```

- `mp.get_context("spawn").Pool(len(jobs))` — a pool of fresh Python processes, one per job (Day 92's way).
- `pool.map(fine_tune, jobs)` — runs fine_tune on every job at once and returns the results in the jobs' order.

SYSTEM DIAGRAM



One general model, four ways to adapt it, and a from-scratch baseline.

Segment 3 — The experiments

demo_sweep and demo_compare: choose on dev, then compare

CODE

```
k = int(np.argmin([r["dev"] for r in res])); best_lr = FT_LRS[k]
print(f"chosen: peak learning rate {best_lr}")
print("Actual output from this lesson's run.")
return best_lr, res[k]
...
jobs = [("from scratch", "scratch", "all", SCRATCH_LR, 1.0, False, 0), ("top block + output only",
    "pretrained", "top", best_lr, 1.0, False, 0),
        ("replay", "pretrained", "all", best_lr, 1.0, True, 0), ("a quarter of the data",
            "pretrained", "all", best_lr, SMALL_FRAC, False, 0)]
res = [full] + run_parallel(jobs)
```

- `np.argmin([...])` — the index of the lowest dev loss: the chosen learning rate.
- `return best_lr, res[k]` — the chosen run is reused as the full fine-tuning row instead of training it again.
- `[full] + run_parallel(jobs)` — list concatenation: the chosen run first, then the four new ones.

demo_samples: speaker lines in generated text

CODE

```
for name, texts in out.items():
    sp = [m for t in texts for m in re.findall(r"^([A-Z][A-Z' ]+):$", t, re.M)]
    share = np.mean([s in new for s in sp]) if sp else 0.0
    print(f"  {name:<20}: {len(sp)} speaker lines, {share:.3f} of them name a speaker new in The
        Tempest")
```

- `re.findall(r"^([A-Z][A-Z' ]+):$", t, re.M)` — re.M makes ^ and $ match at every line, so each 'NAME:' line is found; the group keeps the name.
- `np.mean([s in new for s in sp])` — the share of speaker lines whose name is one of the play's new speakers.

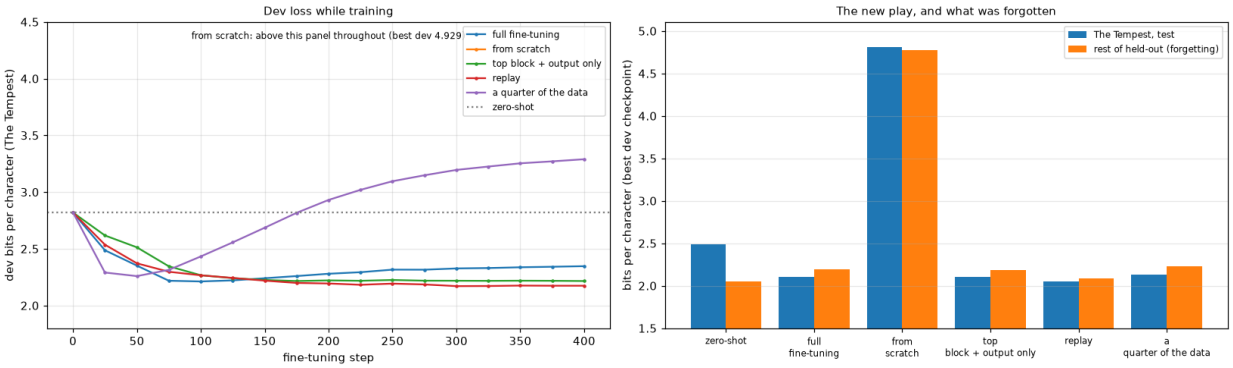
save_model: Day 92's file format, for Day 97

CODE

```
def save_model(full, vocab, cfg):
    flat = {k.replace(".", "__"): v.astype(np.float32) for k, v in d90.w12.leaves(full["P"])}
    np.savez_compressed(os.path.join(HERE, "day96_finetuned_gpt.npz"), vocab=np.array(vocab),
        **flat,
                        **{f"cfg__{k}": v for k, v in cfg.items()})
```

- `k.replace(".", "__")` — 'blocks.0.Wq' becomes 'blocks__0__Wq': one flat name per array in the file.
- `v.astype(np.float32)` — half the file size; Day 93's loader converts back to float64.
- `**{...}` — unpacks a dictionary into keyword arguments: each becomes one named array in the .npz file.

REAL GRAPH



Real graph from the script: dev curves, then test loss and forgetting for each model.

Quick Syntax Glossary — new or reinforced constructs

- `d.setdefault(k, default)` — the value for k, inserting the default first if k is missing.
- `a[:] = b` — write into the existing array in place.
- `pool.map(f, items)` — apply f to every item in parallel processes, results in order.
- `x.copy()` vs `dict(d)` — a new array vs a new dictionary that still holds the same arrays.
- `re.M` — multiline mode: ^ and $ match at line starts and ends.
- `f(**d)` — pass a dictionary's entries as keyword arguments.

End of Day 96 syntax reference. For the experiments and what they showed, see the Day 96 complete guide.